computers

MDPI

Article

PCLLM: An Integrated LLM-Driven System for Automating Desktop Operations via Direct Mouse and Keyboard Control

Zhenqian Wang [1,†], Yi Dong [2,†], Meixia Fu [3,4,*], Jianquan Wang [3], Jie Sun [2], Qu Wang [3,4], Yifan Lu [3], Na Chen [3], Ronghui Zhang [3] and Wen Zhang [1]

[1] School of Computer and Communication Engineering, University of Science and Technology Beijing, Beijing 100083, China; d202410457@xs.ustb.edu.cn (Z.W.); m202410642@xs.ustb.edu.cn (W.Z.)
[2] Shanxi Taihang Laboratory Co., Ltd., Xi'an 030006, China; dong046@foxmail.com (Y.D.); sun-j15@tsinghua.org.cn (J.S.)
[3] Beijing Key Laboratory of Industrial Deterministic Networks and Intelligent Collaborative Control, School of Automation and Electrical Engineering, University of Science and Technology Beijing, Beijing 100083, China; wangjianquan@ustb.edu.cn (J.W.); wangqu@ustb.edu.cn (Q.W.); d202410379@xs.ustb.edu.cn (Y.L.); chenna@ustb.edu.cn (N.C.); zhangrh@ustb.edu.cn (R.Z.)
[4] Shunde Graduate School, University of Science and Technology Beijing, Foshan 528399, China
* Correspondence: mxfu1205@ustb.edu.cn; Tel.: +86-17801095207
† These authors contributed equally to this work.

Abstract

The of personal computer (PC) tasks represents a systems-level challenge that integrates natural language processing, visual perception and mouse–keyboard action control. While existing approaches mainly focus on the application programming interface (API)-based or terminal-based automation, which are incompatible with the majority of applications for the lack of accessible interface. In this article, we propose PCLLM, a novel end-to-end system that automates PC operations by integrating large language models (LLMs) with computer vision techniques to directly control the mouse and keyboard. First, a software knowledge-based prompt engineering method is developed to comprehend software architecture and operational sequences. Second, template matching techniques are integrated for precise element localization, allowing the system to accurately identify and interact. Third, a dual-LLM pipeline is designed to automatically generate the test data, where a questioner LLM generates diverse task commands and the PCLLM executes these tasks, the corresponding process data are recorded automatically for performance evaluation. Finally, PCLLM is further validated through three typically PC applications (Notepad, Wordpad and Calculator), demonstrating its flexible and robust performance towards intelligent PC automation. To evaluate the proposed system, we adopt task completion rate as the primary metric. Experimental results show that PCLLM achieves the highest completion rates of 98.59%, 95.77%, and 52.11% on Notepad for basic, intermediate, and advanced tasks respectively when powered by GPT-4o, outperforming the CogAgent baseline. These results demonstrate the effectiveness of our approach for PC task automation.

Keywords: intelligent systems; human–computer interaction; large language model; PC automation

Check for updates

Academic Editors: Phivos Mylonas, Katia Lida Kermanidis and Manolis Maragoudakis

Received: 28 April 2026
Revised: 27 May 2026
Accepted: 28 May 2026
Published: 30 May 2026

Copyright: © 2026 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license.

1. Introduction

Driven by the growing complexity of software systems, the automation of personal computers (PCs) has become an increasingly important research topic [1] and offers multiple advantages such as improving efficiency. The automation of PCs refers to an algorithm or

model that interacts with the software for task completion, thus improving user efficiency by offloading routine tasks [2]. Consequently, PC task automation holds significant potential for enhancing individual and organizational performance.

In the early stages of research, PC task automation relied on script-based or rule-based methods designed for specific tasks [3]. With the advancement of artificial intelligence, smarter voice assistants such as Siri and Alexa emerged, capable of following human instructions to perform simple assignments on mobile devices. Consequently, interactive environments have been predominantly developed for mobile scenarios [4,5] or web applications [6–10]. However, the voice assistants rely on predefined intents and parameter extraction from user queries which limits their adaptability to novel linguistic patterns or unanticipated contextual nuances. Furthermore, these environments are typically built on the Android ecosystem and designed for relatively simple and narrowly defined tasks.

With the advent of ChatGPT, large language model (LLM)-based automation methods have been broadly explored for tasks automation [11–16]. Additionally, human feedback reinforcement learning has been investigated to refine LLM behavior, leading to more accurate and context-aware interactions in complex scenarios [17]. While some benchmarks [18–21] have been extended to include more diverse operations, existing methods remain insufficient in capturing the full complexity and task diversity inherent in desktop computing environments.

Humans can interact with PCs using only a mouse and keyboard to accomplish a wide variety of complex tasks across multiple software environments. Therefore, equipping LLMs with the ability to accurately control mouse and keyboard represents a more effective and intelligent approach to PC automation. Specifically, a mouse-and-keyboard-based solution offers several advantages, including alignment with human intuition and applicability across virtually all software systems. Gao et al. [22] pioneered an LLM-based actor–critic framework named ASSISTGUI, which enables the automation of complex desktop tasks by parsing graphical user interface (GUI) and employing advanced reasoning techniques. However, ASSISTGUI employed several large neural networks for subtasks, requiring heavy resources while yielding suboptimal performance.

PC operation through mouse and keyboard control requires the model to develop two capabilities. First, the model possesses knowledge of the software being operated, including the hierarchical structure, the function of a specific button and the operation sequence to complete a given task. Second, the control model should be capable of accurately locating the target button or text elements in the GUI environment to perform precise mouse clicks. In this article, we propose a PC automation end-to-end system called PCLLM based on LLM technology. Specifically, the approach uses an LLM to parse user commands and generate corresponding action sequence in text format. Template matching is then employed to accurately locate GUI elements on the PC screen. Finally, the action sequence is converted into actual mouse and keyboard operations using a PC automation library like PyAutoGUI. In addition to PCLLM, we develop a novel method to automate the process of generating test data, which utilizes two LLMs acting as a "questioner" and an "executor". The questioner generates diverse task commands, while the executor performs the tasks and records the corresponding mouse and keyboard actions. Through automated question–execute cycles, the two LLMs jointly produce test data.

This study addresses the following research questions. First, can an LLM-based system effectively automate PC operations by generating mouse and keyboard actions? Second, how do different LLMs (GPT-4o, GPT-4o-mini, Gemini-1.5-Pro, and Gemini-2.0-flash-exp) perform across tasks of varying complexity? The independent variable is the choice of LLM and task complexity (basic/intermediate/advanced), while the dependent variable is the task completion rate. Our hypothesis is that more capable LLMs (e.g., GPT-4o) will achieve

higher completion rates, and that task complexity will negatively impact performance across all models.

The contributions of our article are concluded as follows:

- We propose a novel end-to-end PC task automation framework that leverages LLMs to operate the mouse and keyboard. The experimental results demonstrate the framework's robust performance across three PC applications which are Wordpad, Notepad and Calculator, The experimental results demonstrate the framework's robust performance across Notepad, Wordpad, and Calculator, achieving task completion rates of up to 98.59%, 91.55%, and 84.43% respectively when powered by GPT-4o.
- We present an advanced prompt engineering methodology that systematically integrates three software-related components which are software hierarchical architecture, button functional descriptions and few-shot examples. This prompt significantly enhances the LLM ability on understanding and executing PC tasks.
- A dual-LLM pipeline architecture is presented, in which one model generates diverse task descriptions while another executes and records the corresponding operations, demonstrating the feasibility of fully automated test data generation for PC automation tasks.
- We conduct extensive experimental validation using three standard PC applications (Notepad, WordPad, and Calculator), with quantitative results showing superior performance compared to existing approaches.

2. Related Work

Algorithm-driven PC task automation is an exciting and promising research topic that aims to enhance human productivity by freeing users from complex desktop environments. Existing approaches to PC task automation can be broadly categorized into three branches which are terminal-based methods using tools such as NL2Bash for Ubuntu [23,24], application programming interface (API)-based automation as seen in DroidBot-GPT along with AutoDroid [21,25], and keyboard-and-mouse operation automation exemplified by ASSISTGUI and CogAgent [22,26].

Terminal-based methods offer an advantage, as the input and output of a terminal take the form of text, which is compatible with the language processing ability of LLMs naturally. Lin et al. [23] proposed NL2Bash which translates natural language instructions into Linux command-line operations using LLMs, improving command accuracy and execution efficiency. Similarly, Liu et al. [24] developed a benchmark platform and employed reinforcement learning algorithms to adapt agents to diverse terminal tasks, achieving high completion rates and flexibility. However, terminal-based automation schemes impose limitations on the range of tasks such as file operations and system configuration, and most PC software are designed to operate in a desktop environment, making it impossible to obtain app information in a system terminal.

API-based automation methods directly interact with software-specific functions and publicly available API services to accomplish more complex tasks. Especially with the advent of ChatGPT, API-based methods have received significant attention due to the capability of LLMs to logically organize multiple API calls. Schick et al. [27] developed Toolformer, which was a pioneer of LLM-driven API-based automation models. The authors systematically developed methods for generating training data containing API calls and enabling the model to execute the API calls effectively. Similarly, several studies [21,28] also proposed language models with API calling capabilities. Notably, Qin et al. [28] introduced ToolLLM, which is capable of handling over 16,000 APIs. In general, the API-based automation methods effectively address the limitations of terminal-based methods by enabling direct access to application functions. However, the API-based approaches face

inherent limitations due to API availability constraints, as most desktop software vendors prioritize GUI over programmatic interfaces for security and commercial reasons.

Compared with the previous two branches, GUI-based automation methods offer the most versatility, as they mimic human operation on a PC desktop via keyboard and mouse to accomplish a wide range of complex tasks. For example, Nakano et al. [29] proposed WebGPT which has the ability to simulate human mouse–keyboard operations to interact with a specially designed browser. Gao et al. [22] introduced ASSISTGUI, where an actor–critic framework is used to parse and manipulate GUIs. The experimental results demonstrated that ASSISTGUI achieved significant improvements over previous methods. However, WebGPT and ASSISTGUI demonstrate operational capabilities only within constrained desktop environments. Recently, Hong et al. [26] put forward CogAgent based on visual language models for PC automation. Then Claude 3.5 Sonnet put forward a "computer use" function for UI interactions, exemplifying the growing scholarly engagement with LLM-powered GUI agents. Soon OpenAI proposed a Computer-Using Agent and achieved state-of-the-art performance. However, it should be noted that Claude 3.5 Sonnet and the Computer-Using Agent are not open-source. In addition, many other enterprises have also proposed agents with similar computer-use capabilities, such as OK-Computer from Kimi, Wukong from Alibaba, WorkBuddy from Tencent. The aforementioned GUI models extend the capabilities of automation beyond terminal- and API-based methods by directly engaging with graphical elements, making well-trained model applicable to a broader range of desktop tasks.

The existing approaches mentioned above have advanced PC automation considerably, but several limitations remain. First, the terminal-based methods only support the text paradigm and are unavailable for GUI-driven workflows. Second, due to vendor restrictions, the application scope of API-based approaches is also limited. Finally, open-source models such as CogAgent [26] and ASSISTGUI [22] exhibit suboptimal performance, whereas closed-source models like Claude 3.5 Sonnet and Computer-Using Agent lack technical transparency. To address the challenges mentioned above, we propose PCLLM for PC task automation. The PCLLM leverages the capabilities of text-based LLMs while maintaining architectural simplicity.

3. Methodology

Figure 1 presents the comprehensive system architecture of PCLLM, highlighting its structure with three critical modules which are prompt engineering, LLM and terminal toolkit. First, the prompt engineering is specifically designed to encode software-related information into the LLM. Second, the LLM aims at two primary tasks: understanding user queries and generating action sequences. The action sequences adhere to a rigorously defined syntactic structure to facilitate automated translation into executable keyboard and mouse operations through programmatic interpretation. Then the template matching algorithm is employed to obtain accurate coordinates of mouse click targets. Finally, the system utilizes PyAutoGUI to automatically perform the corresponding mouse operations at precise screen coordinates, thereby completing the translation from linguistic commands to physical interactions.

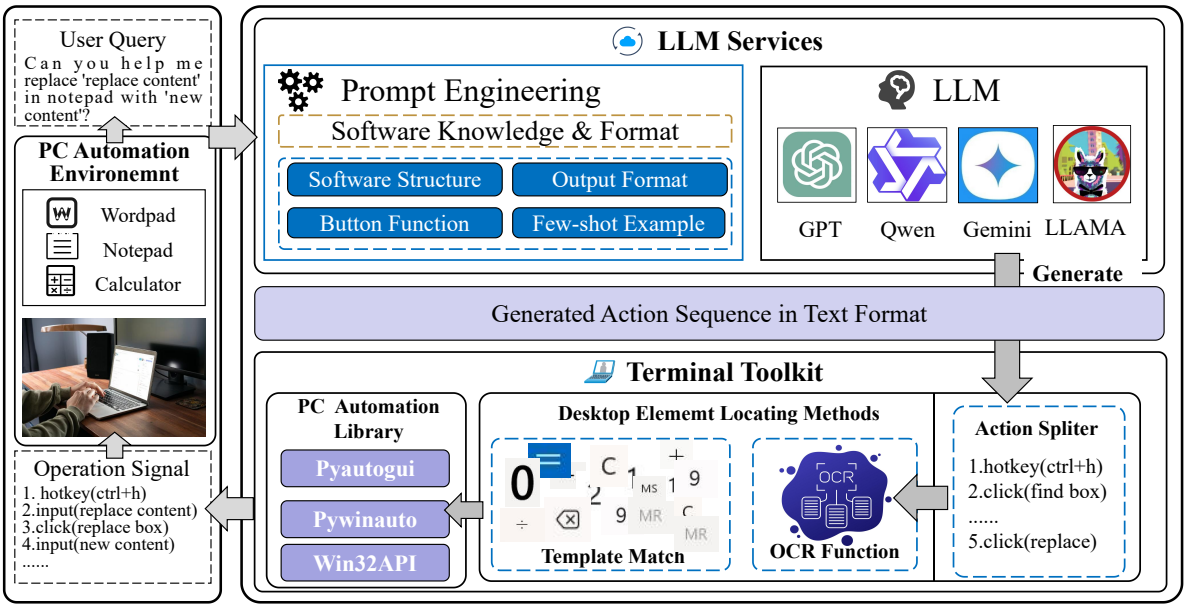


Figure 1. The workflow demonstration for PCLLM.

3.1. Prompt Engineering

Prompt engineering serves as a critical technique to optimize the output quality and operational efficacy of LLMs in the context of PC task automation. In PCLLM, prompt engineering consists of three core components: software hierarchical architecture, button functional descriptions and few-shot examples.

Software Hierarchical Structure. The software hierarchical structure indicates which function would expand when a button is clicked. In PCLLM, the software hierarchical structure provides two advantages: providing prior knowledge of software and helping the LLM navigate the GUI. Take Notepad as an example, the LLM learns that activating the "Save As" function in Notepad requires first clicking on the "File" menu. In our design, the hierarchy is organized into a Python dictionary format to ensure the structure is easily parsed by the LLM.

Button Functionality and Hotkeys. In our work, button functionality and hotkeys are critical in enhancing LLM performance on PC automation tasks. Specifically, button functionality provides a short description of each software component. The hotkey refers to the keyboard shortcut that can directly invoke corresponding functions. This design brings a couple of benefits. First, the detailed button functionality description helps distinguish between similar functions, such as "Save" and "Save as" in Notepad. In addition, hotkeys offer shortcuts to directly invoke related functions thus improving operational efficiency significantly.

Few-Shot Examples. Few-shot examples are efficient methods for improving LLM performance in relevant downstream tasks. Based on the few-shot examples, the LLM is capable of completing similar tasks with high accuracy. In our work, we provide examples of some simple tasks. Each example provides a natural language instruction paired with the corresponding action sequence (e.g., "Search for 'invoice' in the document" - hotkey(Ctrl+f)-input(invoice)-press(Enter)). Experimental results demonstrate that few-shot examples most significantly enhance model performance on test tasks.

Other Content. Finally, to better parse the action sequence into individual operation steps, the LLM is required to generate strictly formatted output. In our work, the prompt requires all actions to conform to a predefined form such as click(File) and hotkey(Ctrl+S). Each pattern comprises an action type and associated parameters of action as specified in

Table 1. Then all actions should be separated by a "-" symbol to indicate the sequence (e.g., click(File)-click(Save As)-input(report.txt)). Additionally, the prompt explicitly excludes natural language explanations or extraneous text from the output to ensure the result is directly parsed and executed by automation scripts.

Table 1. The action space of an LLM model.

Function	Parameter	Instance
moveTo	button	MoveTo (File)
click	button	Click (File)
doubleclick	button	Doubleclick (File)
scroll	times	Scroll (10)
write	message	Write (Hello)
keyDown	key	KeyDown (UP)
keyUp	key	KeyUP (UP)
hotkey	* keys	Hotkey (Ctrl + c)

3.2. GUI Element Localization via Template Matching

After obtaining the action sequence of a specific task, the next objective is to accurately locate GUI elements. For simplicity, we utilize the template matching technique to directly obtain corresponding coordinates of the clickable targets.

Template Matching. For non-text UI components, a template matching approach is employed based on normalized cross-correlation. Given template image T and screen region S, the similarity is shown in Equation (1), where $T(x', y')$ is the template image, $I(x, y)$ is the source image and $R(x, y)$ is the correlation result matrix. The detection confidence score at position (x, y) is given by Equation (2). Then the confidence score is compared with a user-defined threshold τ. If the confidence score exceeds the threshold, $C(x, y)$ is returned by template matching:

$$R(x,y) = -\frac{\sum_{x',y'}(T(x',y') \cdot I(x+x', y+y'))}{\sqrt{\sum_{x',y'} T(x',y')^2 \cdot \sum_{x',y'} I(x+x', y+y')^2}} \tag{1}$$

$$C(x,y) = \max(R(x,y)) \in [0,1] \tag{2}$$

3.3. Two LLMs for Automated Test Data Generation

To minimize manual effort in generating test data, a dual-LLM architecture is proposed to streamline the generation of test data as shown in Figure 2. The framework consists of two specialized LLMs, a questioner model that generates diverse task descriptions and an executor model that performs the corresponding GUI operations. The collaborative interaction between the two models establishes a closed-loop testing system that efficiently produces evaluation data.

In detail, the questioner is responsible for generating specific instructions in natural language. In our work, the tasks range from basic operations to complex tasks. Take Notepad as an example, where the basic operations include creating a new document, opening a new tab, etc., and the complex tasks include file management and text formatting problems. In addition, the questioner is encouraged to introduce systematic variations in task parameters to ensure comprehensive test coverage.

The executor receives task descriptions generated by the questioner and outputs corresponding action sequences. All sequences strictly comply with the prescribed structure to maintain interoperability with the automated execution framework. During execution, the monitoring system captures detailed procedural records containing all mouse and keyboard actions with precise step numbers and visual information through sequential screenshots.

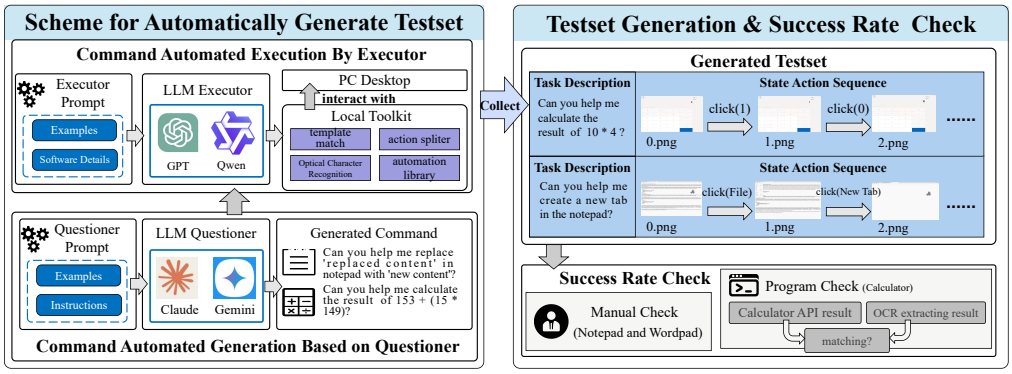


Figure 2. Structure of two LLMs for automatically generating dataset.

3.4. Materials

All experiments were conducted on a Windows 11 operating system. The screen resolution was fixed at 1920 × 1080 with 100% scaling to ensure consistent GUI layout. The automation framework was implemented in Python 3.10. Key libraries included PyAutoGUI (version 0.9.54) for mouse and keyboard control and OpenCV-Python (version 4.8.1) for template matching. All large language model inferences were performed via official cloud application programming interfaces; no local graphics processing unit was required. The specific LLMs and their API versions are detailed in Section 4.2. The three target applications were the native Windows 11 versions of Notepad, WordPad, and Calculator.

4. Experiment

4.1. Experimental Environment

Our experiment employs Notepad, WordPad, and Calculator as representative cases for comprehensive performance analysis of PCLLM. Specifically, Notepad and WordPad are mainly used to evaluate adaptability and generalizability of our method. The Calculator is used to assess the logical ability of the LLM because the memory-related function requires the model to fully understand how to use the memory function to store and record the intermediate values of the computation. An example of the running process of our system is shown in Figure 3.

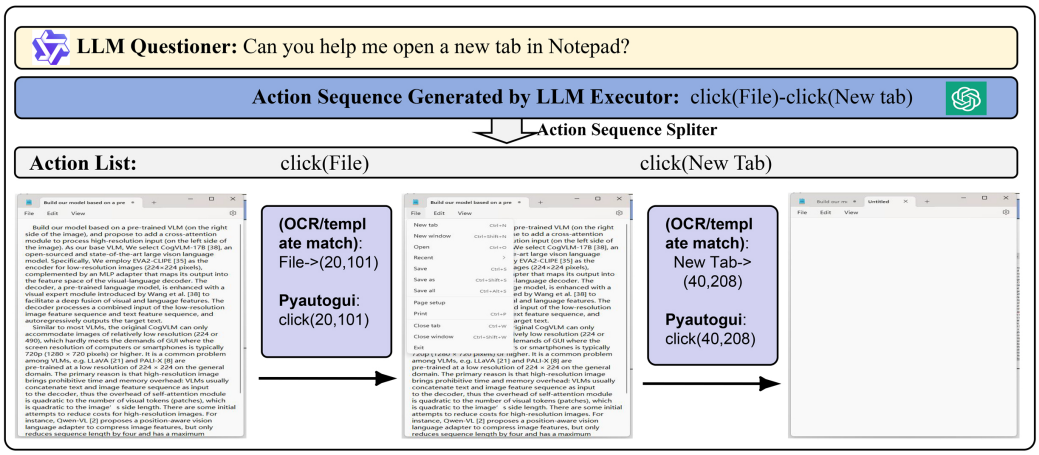


Figure 3. An example of our proposed PC automation operation scheme.

All template images are manually pre-captured for each software application in this work. For each clickable UI element, one template image is stored. During execution, if template matching fails (confidence below threshold τ, which is set as 0.95 in our experiment), the system skips the current action and proceeds to the next one.

4.2. Baseline and Comparison

In this part, experiments are designed to evaluate capability of LLMs in understanding human instruction and performing complex desktop operations. In addition, another target is to select the optimal base model for our framework. Specifically, we choose four LLMs: GPT-4o, GPT-4o-mini, Gemini-1.5-Pro and Gemini-2.0-flash-exp to evaluate our scheme. These models are selected for three reasons. OpenAI and Google are two of the leading LLM providers, making their models representative of the state of the art. The selected models span different capability levels, from flagship to lightweight and efficiency-oriented designs, allowing a systematic evaluation across diverse LLM backbones. Additionally, all selected models provide public APIs, which ensures the reproducibility of our experiments. It should be noted that each model is evaluated under the same task scenarios to ensure the consistency and comparability of experimental results and. For each LLM and each application, we conducted 216 tasks in total, with 72 tasks for each complexity level (basic, intermediate, and advanced).

The test tasks are stratified into three complexity levels to systematically evaluate model performance across varying difficulty spectra: basic, intermediate, and advanced. For Notepad and WordPad, basic tasks typically include straightforward actions such as opening a new tab, saving the current file with a new filename, or pasting the clipboard content into the cursor position in the software. Intermediate tasks involve the combination of two basic tasks such as "open a new tab and save it as 'new-file.txt'." The advanced tasks are the combination of three to five basic operations across multiple basic tasks. For the Calculator application, basic tasks involve fundamental arithmetic operations on binomial expressions (e.g., "151 plus 31 = ?"), while intermediate tasks require evaluating composite arithmetic expressions where at least one term is an arithmetic operation rather than a single number (e.g., "15 plus (101 multiplied by 21) = ?"). Advanced tasks involve composite arithmetic expressions where each term is an expression such as "(13 plus 21) divided by (116 multiplied by 13) = ?" or "(13 plus 21) divided by (116 multiplied by 13) multiplied by (2 minus 5) = ?". The intermediate and advanced tasks of the calculator require the use of memory-related operations, which can be challenging even for human users.

We choose the task completion rate as an evaluation metric to assess model performance. The task completion rate is defined as the number of tasks completed divided by the total number of tasks. In addition, for different software, different methods are utilized to calculate the task completion rate. Specifically, for applications like Notepad and WordPad, a manual annotation method is applied. The human evaluators manually assess whether the task is completed by observing the action sequence generated by the LLM and corresponding screenshot transitions. For the Calculator, a two-stage evaluation method is developed to automatically judge whether the task is finished correctly. First, the mathematical expression is calculated by a high-precision calculator API. Then the last screenshot from the sequence is recognized using the optical character recognition (OCR) technique to get the mathematical result produced by the LLM action sequence. Finally, the success rate can be calculated by the percentage of trials where OCR-extracted results match API-based output within the preset floating-point tolerance.

The corresponding experimental results on the three software applications are shown in Figures 4–6. First, the results demonstrate a negative correlation between task complexity and LLM operation accuracy. For example, the success rate of GPT-4o falls from 0.98 on basic tasks to 0.52 on advanced tasks on Notepad as shown in Figure 4. On the Calculator Gemini-1.5-pro-latest drops from 0.90 on basic tasks to 0.49 on advanced tasks as shown in Figure 5. The results suggest that as the length of operation sequence grows, the ability of the LLM to accurately predict the subsequent action diminishes, revealing limitations in handling complex operational workflows. Second, GPT-4o achieves the highest success

rate on Notepad and WordPad as shown in Figure 4 (e.g., 0.96 basic, 0.87 intermediate, 0.28 advanced on Notepad for Gemini-1.5-pro vs. 0.98/0.95/0.52 for GPT-4o), whereas Gemini-1.5-pro exhibits superior performance on Calculator as shown in Figure 5 (0.89 on basic, 0.80 on intermediate, 0.49 on advanced, versus 0.84/0.50/0.56 of GPT-4o). The phenomenon suggests that the capabilities of LLMs are dependent on task-specific domains. GPT-4o excels in daily tasks on PC systems and Gemini-1.5-pro demonstrates stronger logical reasoning ability.

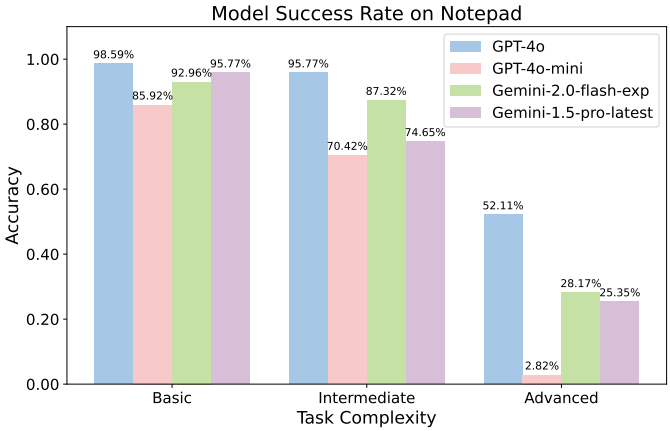


Figure 4. LLMs performance on Notepad.

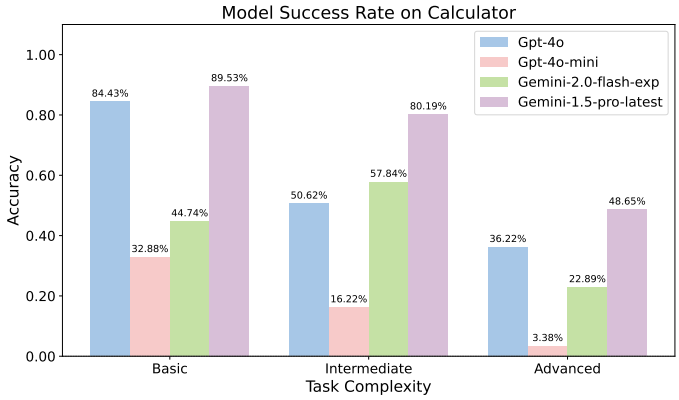


Figure 5. LLMs performance on Calculator.

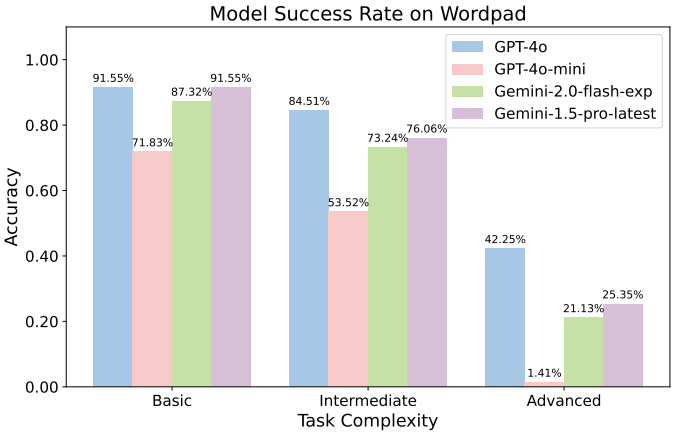


Figure 6. LLMs performance on Wordpad.

To verify that the observed performance differences are not due to random chance, we conducted Kruskal–Wallis tests on the task-level success/failure data (216 tasks per model per application). Details are given in Table 2. The results showed significant differences

among the four LLMs across all three applications ($p < 0.001$ for Notepad and Calculator; $p = 0.003$ for WordPad). Post hoc Mann–Whitney U tests with Bonferroni correction revealed that GPT-4o significantly outperformed all other models on Notepad and WordPad ($p < 0.05$), and also outperformed GPT-4o-mini and Gemini-2.0-flash-exp on Calculator ($p < 0.01$). Gemini-1.5-Pro showed competitive performance on Calculator, while no significant difference was found between Gemini-1.5-Pro and Gemini-2.0-flash-exp on Notepad and WordPad ($p > 0.05$). These results confirm that the performance gaps are statistically reliable.

Table 2. Statistical analysis results of pairwise model comparisons.

Application	Comparison	H-Test *p*	MW *p*	Sig.
Notepad	GPT-4o vs. GPT-4o-mini	<0.001	<0.001	***
	GPT-4o vs. Gemini-1.5-Pro		<0.001	***
	GPT-4o vs. Gemini-2.0-flash-exp		0.002	**
	GPT-4o-mini vs. Gemini-1.5-Pro		0.011	*
	GPT-4o-mini vs. Gemini-2.0-flash-exp		<0.001	***
	Gemini-1.5-Pro vs. Gemini-2.0-flash-exp		0.357	n.s.
Calculator	GPT-4o vs. GPT-4o-mini	<0.001	<0.001	***
	GPT-4o vs. Gemini-1.5-Pro		<0.001	***
	GPT-4o vs. Gemini-2.0-flash-exp		0.002	**
	GPT-4o-mini vs. Gemini-1.5-Pro		<0.001	***
	GPT-4o-mini vs. Gemini-2.0-flash-exp		<0.001	***
	Gemini-1.5-Pro vs. Gemini-2.0-flash-exp		<0.001	***
WordPad	GPT-4o vs. GPT-4o-mini	0.003	<0.001	***
	GPT-4o vs. Gemini-1.5-Pro		0.018	*
	GPT-4o vs. Gemini-2.0-flash-exp		0.008	**
	GPT-4o-mini vs. Gemini-1.5-Pro		0.204	n.s.
	GPT-4o-mini vs. Gemini-2.0-flash-exp		0.330	n.s.
	Gemini-1.5-Pro vs. Gemini-2.0-flash-exp		0.768	n.s.

*** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$, n.s. = not significant.

In addition to the closed-source LLMs like GPT, several Qwen-series open-source language models are also tested as the core in our framework. The open-source language models from the Qwen family offer benefits such as lightweight design and lower operation cost, which make them suitable for use in limited resource environments. However, the open-source models' performance on complex tasks is inferior to closed-source models such as GPT series and Gemini Series. Hence, the trade-off between model capacity and hardware performance should be carefully considered. Specifically, the Qwen models selected for our experiments are Qwen2.5-7B-Instruct, Qwen2.5-14B-Instruct and Qwen2.5-32B-Instruct. And the Qwen models are tested on Notepad and Calculator respectively. The experimental results are shown in Tables 3 and 4.

Two conclusions can be drawn from Tables 3 and 4. First, the performance of closed-source models significantly outperforms open-source models, especially on advanced tasks. Specifically, the task completion rates of GPT-4o on Notepad are 98.59%, 95.77%, and 52.11% from basic to significantly outperforming open-source models, especially on advanced tasks. In comparison, the success rates of Qwen2.5-32B-Instruct drop to 91.55%, 69.01% and 45.07% respectively. Similarly, the same trend is observed on Calculator application between Gemini-1.5-pro-latest and Qwen-series models in Table 4. In addition, the performance improves significantly as the parameter size of the models increases in the Qwen2.5 family, which is easily observed from Tables 3 and 4. In conclusion, the results highlight the trade-off between resource efficiency and model capability on complicated user commands.

Table 3. Performance of different Qwen models on Notepad tasks.

Model	Task Complexity	Success Rate (%)
GPT-4o	basic	98.59
	intermediate	95.77
	advanced	52.11
Qwen2.5-32B-Instruct	basic	91.55
	intermediate	69.01
	advanced	45.07
Qwen2.5-14B-Instruct	basic	85.91
	intermediate	64.79
	advanced	36.62
Qwen2.5-7B-Instruct	basic	56.34
	intermediate	53.52
	advanced	28.17

Table 4. Performance of different Qwen models on Calculator tasks.

Model	Task Complexity	Success Rate (%)
Gemini-1.5-pro-latest	basic	89.53
	intermediate	80.19
	advanced	48.62
Qwen2.5-32B-Instruct	basic	70.21
	intermediate	19.84
	advanced	16.90
Qwen2.5-14B-Instruct	basic	42.23
	intermediate	12.68
	advanced	8.51
Qwen2.5-7B-Instruct	basic	87.32
	intermediate	12.67
	advanced	2.82

4.3. Ablation Study

An ablation study is also conducted on the different parts of the prompts. The experiment is designed to identify how the inclusion of each part of the prompt impacts the performance of LLMs. The original prompt used in our scheme includes four parts: the hierarchical structure of the software, the functionality of each clickable button, the hotkey usage and the few-shot examples for specific tasks. In the ablation study, the experiment is set up with four configurations: (1) removing the hierarchical structure of software, (2) omitting the few-shot examples for task operation, (3) excluding the functionality of the clickable button, and (4) using only the task description, without any supplementary prompts. Regarding the experimental setup, two LLMs that possess the best performance in the baseline experiments on Notepad are evaluated to ensure the reliability of the ablation study. Each ablation evaluation includes 72 individual test tasks.

Based on the experimental results shown in Table 5, two key conclusions are drawn regarding the model performance. First, prompt engineering plays an essential role in enhancing the LLM ability to interact with software, as the performance drops sharply when only basic task descriptions are utilized. For GPT-4o, the task completion rate plummeted to 47.89%, 7.04%, and 0% for basic, intermediate, and advanced tasks respectively without supplementary prompts, compared to 98.59%, 95.77%, and 52.11% with the full prompt. The same phenomenon is observed for Gemini-2.0-flash-exp. Second, the hierarchical software architecture and detailed operational steps (few-shot examples) are of comparable importance, while button functionalities and hotkey descriptions play a relatively minor

role. For instance, removing the hierarchical structure caused the performance of GPT-4o on advanced tasks to drop sharply from 52.11% to 21.13%, and omitting operational steps led to an even more dramatic decline to 15.49%. In contrast, excluding button functionalities has a mild impact, with the advanced task rate decreasing only slightly from 52.11% to 49.30%. The same pattern is observed for Gemini-2.0-flash-exp. The hierarchical structure ablation reduced advanced task performance from 28.17% to 5.63%, while the absence of operational steps lowered it to 12.68%. Button functionality and hotkey usage methods exclusion resulted in a smaller reduction to 22.54%. The comprehensive ablation study results are systematically presented in Table 5, demonstrating the contribution of each component.

Table 5. Task completion rate for different ablation parts and models.

Model	Ablation Part	Task Complexity	Complication Rate (%)
GPT-4o	full prompt	basic	98.59
		intermediate	95.77
		advanced	52.11
	remove architecture	basic	94.37
		intermediate	67.61
		advanced	21.13
	no button function	basic	98.59
		intermediate	97.18
		advanced	49.30
	no hotkey	basic	97.22
		intermediate	90.27
		advanced	48.61
	no few-shot example	basic	91.55
		intermediate	59.15
		advanced	15.49
	only basic desc	basic	47.89
		intermediate	7.04
		advanced	0.00
Gemini-2.0-flash-exp	full prompt	basic	92.96
		intermediate	87.32
		advanced	28.17
	remove architecture	basic	92.96
		intermediate	76.06
		advanced	5.63
	no button function	basic	94.37
		intermediate	78.87
		advanced	22.54
	no hotkey	basic	91.66
		intermediate	77.78
		advanced	25.00
	no few-shot example	basic	95.77
		intermediate	77.46
		advanced	12.68
	only basic desc	basic	35.21
		intermediate	4.23
		advanced	0.00

From the experimental results presented above, it can be concluded that the specially designed prompt significantly enhances LLM performance on PC automation downstream tasks. In addition, the results also provide valuable insights for refining prompt engineering

techniques for future research, aiming at helping LLMs to better interact with desktop environments in complex GUI scenarios.

4.4. Comparison with CogAgent

A comparison experiment is conducted between PCLLM and CogAgent which is a leading open-source model in the field of GUI-based PC task automation. Specifically, we adopt the CogAgent-9B variant. The evaluation uses the same 216 Notepad tasks (72 per complexity level: basic, intermediate, and advanced) as in the main experiments. CogAgent is deployed on a GPU server (NVIDIA H800) and accessed via its API. Task completion is determined by the final screen state following the same protocol used for PCLLM. In our experiments, CogAgent is integrated into the same test environment to evaluate its performance alongside the LLM-based automation system proposed in this paper. Details of the experimental results are shown in Table 6.

Table 6. Task success rates on Notepad for different schemes.

Scheme	Task Type	Success Rate (%)
CogAgent	Basic	45.07
	Intermediate	8.45
	Advanced	0.00
Our scheme, (GPT-4 as decision model)	Basic	98.59
	Intermediate	95.77
	Advanced	52.11
Our scheme, (Qwen2.5-7B-Instruct as decision model)	Basic	56.34
	Intermediate	53.52
	Advanced	28.17

The results show that PCLLM outperforms CogAgent by 53.52% on basic tasks. It should be noted that CogAgent has 9B parameters. Even when integrating the Qwen2.5-7B-Instruct into our framework, performance of PCLLM still exceeds CogAgent across diverse task types, which is attributed to our innovative approach in prompt engineering and the integration of template matching techniques for accurate GUI interaction. In contrast, CogAgent's performance decreases significantly as the task complexity increases. Consequently, although CogAgent is a robust baseline solution, our experimental findings highlight how flexible system architecture and prompt design can deliver superior reliability and accuracy in automating complex tasks across varied desktop environments.

5. Discussion

Our experimental results demonstrate three key findings. First, LLM-based PC automation is feasible, with PCLLM achieving high task completion rates across Notepad, WordPad, and Calculator. Second, model capability significantly impacts performance: GPT-4o consistently outperforms other LLMs, particularly on complex tasks. Third, prompt engineering plays a critical role as evidenced by the ablation study where removing prompt components caused substantial performance drops.

Regarding our research questions, we find that: (1) an LLM-based system can effectively automate PC operations by generating mouse and keyboard actions, and (2) task complexity negatively impacts all models, though more capable LLMs (e.g., GPT-4o) maintain higher success rates on complex tasks. These results confirm our hypothesis.

For users and practitioners, PCLLM offers a practical solution for automating repetitive desktop tasks without requiring API access or terminal interfaces. The system can benefit knowledge workers, accessibility tools, and software testing scenarios. However, current limitations—manual prompt engineering, template sensitivity to resolution changes, and

inability to handle pop-up dialogs—mean that deployment is best suited for controlled environments with stable UI layouts.

Despite achieving high accuracy, our approach has several limitations. First, it requires manually crafted prompts and pre-captured button templates for each application, lacking generalizability across different software. Second, NCC-based template matching is sensitive to resolution changes and window scaling, with no dynamic template update mechanism. Third, the LLM generates the entire action sequence upfront, preventing it from handling unexpected screen changes such as pop-up dialogs. Fourth, the approach struggles with complex applications and web browsing tasks where prompt engineering becomes infeasible.

To address these limitations, future work will focus on two directions. First, for robust GUI localization, we plan to replace NCC-based matching with multi-scale matching, adaptive thresholds, and lightweight object detectors such as YOLOv8 [30] and YOLO-TCS [31]. Feature-based retrieval methods, such as feature-aware multi-head self-attention hashing [32] and adaptive pyramid vision transformer (Adaptive-PVT) [33] also offer promising directions. Second, to enable step-by-step decision-making, we plan to integrate multimodal LLMs that can perceive screen states and handle unexpected changes.

6. Conclusions

PC automation is an important research topic and has the potential to free human effort for more innovative work. In this article, we propose a novel PC automation framework named PCLLM that consists of three parts: prompt engineering, LLM and PC automation tools like PyAutoGUI. Specifically, the prompt engineering provides software-related knowledge to the LLM to improve model performance. The LLM generates correct action sequences to accurately accomplish user instructions. Finally, the PC automation tools are responsible for converting action sequences into mouse and keyboard operations on the computer. We test PCLLM on three PC software applications. Experimental results demonstrate the feasibility of our scheme. Specifically, PCLLM achieves task completion rates of up to 98.6%, 95.8%, and 52.1% on Notepad for basic, intermediate, and advanced tasks respectively when powered by GPT-4o, outperforming both other LLMs and the CogAgent baseline. Our future work will focus on enhancing the accuracy and robustness of the PCLLM framework.

Author Contributions: Conceptualization, Z.W. and Y.D.; methodology, Z.W. and Y.D.; software, Z.W., Y.D. and J.S.; validation, Z.W., Y.D., M.F., J.W., Q.W., Y.L., N.C., R.Z. and W.Z.; formal analysis, Z.W. and Y.D.; investigation, Z.W. and Y.D.; resources, M.F. and J.W.; data curation, Z.W., Y.D. and Q.W.; writing—original draft preparation, Z.W. and Y.D.; writing—review and editing, M.F., J.W., J.S., Q.W., Y.L., N.C., R.Z. and W.Z.; visualization, Z.W. and Y.D.; supervision, M.F. and J.W.; project administration, M.F.; funding acquisition, M.F., Q.W. and J.W. All authors have read and agreed to the published version of the manuscript.

Funding: This work was supported in part by National Science and Technology Major Project (2025ZD1602303), National Natural Science Foundation of China (U25A20433, 92567203, 42401521), Joint Research Fund for Beijing Natural Science Foundation and Haidian Original Innovation (L232001), Henan Key Research and Development Program (241111320700), GuangDong Basic and Applied Basic Research Foundation (2024A1515011866, 2024A1515011480, 2025A1515011300), Central Guidance on Local Science and Technology Development Fund of ShanXi Province (YDZJSX20231D005, YDZJSX2024B017), Science and Technology Innovation Program of Xiongan New Area under Grant 2025XAGG0028, National Key Research and Development Program of China 2023YFF0905903.

Data Availability Statement: The data presented in this study is available on request from the corresponding author due to the privacy reasons.

Conflicts of Interest: Authors Yi Dong and Jie Sun are employed by the company Shanxi Taihang Laboratory Co., Ltd. The remaining authors declare that the research was conducted in the absence of any commercial or financial relationships that could be construed as a potential conflict of interest.

References

1. Zhang, C.; He, S.; Qian, J.; Li, B.; Li, L.; Qin, S.; Kang, Y.; Ma, M.; Liu, G.; Lin, Q.; et al. Large language model-brained gui agents: A survey. *arXiv* **2024**, arXiv:2411.18279.
2. Zimmermann, D.; Koziolek, A. Automating gui-based software testing with gpt-3. In Proceedings of the 2023 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW), Dublin, Ireland, 16–20 April 2023; pp. 62–65.
3. Fabio, P. End User Development: Survey of an Emerging Field for Empowering People. *Isrn Softw. Eng.* **2013**, *2013*, 532659.
4. Schneider, S.; Werner, S.; Khalili, R.; Hecker, A.; Karl, H. mobile-env: An open platform for reinforcement learning in wireless mobile networks. In Proceedings of the NOMS 2022–2022 IEEE/IFIP Network Operations and Management Symposium, Budapest, Hungary, 25–29 April 2022; pp. 1–3.
5. Collins, E.; Neto, A.; Vincenzi, A.; Maldonado, J. Deep reinforcement learning based android application gui testing. In Proceedings of the XXXV Brazilian Symposium on Software Engineering, Joinville, Brazil, 27 September–1 October 2021; pp. 186–194.
6. Deng, X.; Gu, Y.; Zheng, B.; Chen, S.; Stevens, S.; Wang, B.; Sun, H.; Su, Y. Mind2web: Towards a generalist agent for the web. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 28091–28114.
7. Pasupat, P.; Jiang, T.S.; Liu, E.; Guu, K.; Liang, P. Mapping natural language commands to web elements. In Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing, Brussels, Belgium, 31 October–4 November 2018; pp. 4970–4976.
8. Shi, T.; Karpathy, A.; Fan, L.; Hernandez, J.; Liang, P. World of bits: An open-domain platform for web-based agents. In Proceedings of the International Conference on Machine Learning, Sydney, Australia, 6–11 August 2017; pp. 3135–3144.
9. Yao, S.; Chen, H.; Yang, J.; Narasimhan, K. Webshop: Towards scalable real-world web interaction with grounded language agents. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 20744–20757.
10. Zhou, S.; Xu, F.F.; Zhu, H.; Zhou, X.; Lo, R.; Sridhar, A.; Cheng, X.; Ou, T.; Bisk, Y.; Fried, D.; et al. Webarena: A realistic web environment for building autonomous agents. In Proceedings of the International Conference on Learning Representations, Vienna, Austria, 7–11 May 2024; Volume 2024, pp. 15585–15606.
11. Shen, Y.; Song, K.; Tan, X.; Li, D.; Lu, W.; Zhuang, Y. Hugginggpt: Solving ai tasks with chatgpt and its friends in hugging face. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 38154–38180.
12. Wei, J.; Wang, X.; Schuurmans, D.; Bosma, M.; Xia, F.; Chi, E.; Le, Q.V.; Zhou, D. Chain-of-thought prompting elicits reasoning in large language models. *Adv. Neural Inf. Process. Syst.* **2022**, *35*, 24824–24837.
13. Pan, Y.; Kong, D.; Zhou, S.; Cui, C.; Leng, Y.; Jiang, B.; Liu, H.; Shang, Y.; Zhou, S.; Wu, T.; et al. Webcanvas: Benchmarking web agents in online environments. *arXiv* **2024**, arXiv:2406.12373.
14. Yang, J.; Zhang, H.; Li, F.; Zou, X.; Li, C.; Gao, J. Set-of-Mark Prompting Unleashes Extraordinary Visual Grounding in GPT-4V. *arXiv* **2023**, arXiv:2310.11441.
15. Yan, A.; Yang, Z.; Zhu, W.; Lin, K.; Li, L.; Wang, J.; Yang, J.; Zhong, Y.; McAuley, J.; Gao, J.; et al. Gpt-4v in wonderland: Large multimodal models for zero-shot smartphone gui navigation. *arXiv* **2023**, arXiv:2311.07562.
16. Touvron, H.; Lavril, T.; Izacard, G.; Martinet, X.; Lachaux, M.A.; Lacroix, T.; Rozière, B.; Goyal, N.; Hambro, E.; Azhar, F.; et al. Llama: Open and efficient foundation language models. *arXiv* **2023**, arXiv:2302.13971.
17. Delaflor, M.; Gendron, C.; Toxtli, C.; Li, W.; Delgado-Solórzano, C.T. ReActIn: Infusing Human Feedback into Intermediate Prompting Steps of Large Language Model. In Proceedings of the AHFE International, San Francisco, CA, USA, 20–24 July 2023.
18. Burns, A.; Arsan, D.; Agrawal, S.; Kumar, R.; Saenko, K.; Plummer, B.A. A dataset for interactive vision-language navigation with unknown command feasibility. In *Proceedings of the European Conference on Computer Vision*; Springer: Berlin/Heidelberg, Germany, 2022; pp. 312–328.
19. Li, Y.; He, J.; Zhou, X.; Zhang, Y.; Baldridge, J. Mapping natural language instructions to mobile UI action sequences. In Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, Online, 5–10 July 2020; pp. 8198–8210.
20. Rawles, C.; Li, A.; Rodriguez, D.; Riva, O.; Lillicrap, T. Androidinthewild: A large-scale dataset for android device control. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 59708–59728.
21. Wen, H.; Li, Y.; Liu, G.; Zhao, S.; Yu, T.; Li, T.J.J.; Jiang, S.; Liu, Y.; Zhang, Y.; Liu, Y. AutoDroid: LLM-powered Task Automation in Android. In Proceedings of the 30th Annual International Conference on Mobile Computing and Networking (MobiCom 2024), Washington, DC, USA, 30 September–4 October 2024.
22. Gao, D.; Ji, L.; Bai, Z.; Ouyang, M.; Li, P.; Mao, D.; Wu, Q.; Zhang, W.; Wang, P.; Guo, X.; et al. ASSISTGUI: Task-Oriented Desktop Graphical User Interface Automation. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), Seattle, WA, USA, 16–22 June 2024.

23. Lin, X.V.; Wang, C.; Zettlemoyer, L.; Ernst, M.D. NL2Bash: A Corpus and Semantic Parser for Natural Language Interface to the Linux Operating System. In Proceedings of the Eleventh International Conference on Language Resources and Evaluation (LREC 2018), Miyazaki, Japan, 7–12 May 2018.
24. Liu, X.; Yu, H.; Zhang, H.; Xu, Y.; Lei, X.; Lai, H.; Gu, Y.; Ding, H.; Men, K.; Yang, K.; et al. AgentBench: Evaluating LLMs as Agents. In Proceedings of the International Conference on Learning Representations (ICLR), Vienna, Austria, 7–11 May 2024.
25. Wen, H.; Wang, H.; Liu, J.; Li, Y. Droidbot-gpt: Gpt-powered ui automation for android. *arXiv* **2023**, arXiv:2304.07061.
26. Hong, W.; Wang, W.; Lv, Q.; Xu, J.; Yu, W.; Ji, J.; Wang, Y.; Wang, Z.; Dong, Y.; Ding, M.; et al. Cogagent: A visual language model for gui agents. In Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, Seattle, WA, USA, 16–22 June2024; pp. 14281–14290.
27. Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Hambro, E.; Zettlemoyer, L.; Cancedda, N.; Scialom, T. Toolformer: Language models can teach themselves to use tools. *Adv. Neural Inf. Process. Syst.* **2023**, *36*, 68539–68551.
28. Qin, Y.; Liang, S.; Ye, Y.; Zhu, K.; Yan, L.; Lu, Y.; Lin, Y.; Cong, X.; Tang, X.; Qian, B.; et al. ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs. In Proceedings of the International Conference on Learning Representations (ICLR) , Vienna, Austria, 7–11 May 2024.
29. Nakano, R.; Hilton, J.; Balaji, S.; Wu, J.; Ouyang, L.; Kim, C.; Hesse, C.; Jain, S.; Kosaraju, V.; Saunders, W.; et al. Webgpt: Browser-assisted question-answering with human feedback. *arXiv* **2021**, arXiv:2112.09332.
30. Zhao, N. Enhancing object detection with yolov8 transfer learning: A voc2012 dataset study. In Proceedings of the International Conference Pattern Recognition Applications and Methods (ICPRAM), Rome, Italy, 24–26 February 2024.
31. Yu, X.; Zhao, X. YOLO-TCS: An enhanced multi-scale network for traffic sign detection integrating multi-level feature fusion and attention. *Multimed. Syst.* **2026**, *32*, 110. [CrossRef]
32. Jiang, H.; Peng, Y.; Li, R.; Peng, Z. Feature-aware multi-head self-attention hashing for Chinese ancient document image retrieval. *Appl. Soft Comput.* **2026**, *193*, 114770. [CrossRef]
33. Zhu, V.; Ji, Z.; Guo, D.; Wang, P.; Xia, Y.; Lu, L.; Ye, X.; Zhu, W.; Jin, D. Low-rank continual pyramid vision transformer: Incrementally segment whole-body organs in CT with light-weighted adaptation. In *Proceedings of the International Conference on Medical Image Computing and Computer-Assisted Intervention*; Springer: Berlin/Heidelberg, Germany, 2024; pp. 371–381.

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.